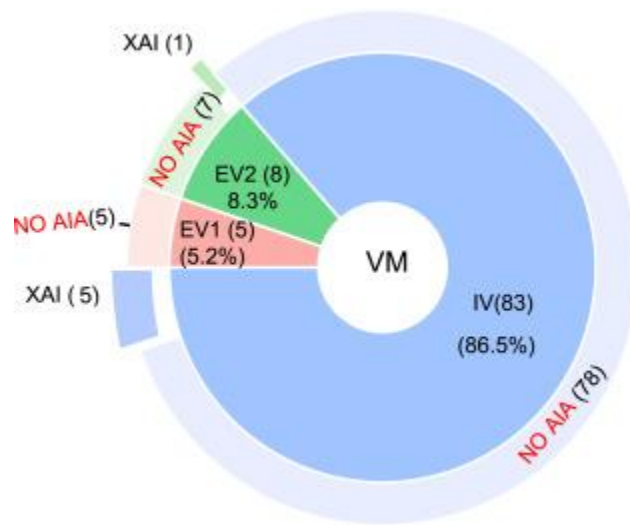


Background

Ovarian cancer is one of the deadliest gynecologic malignancies, responsible for 5% of cancer deaths in women and ranking as the fifth leading cause of cancer-related deaths. Its high mortality rate stems from subtle, nonspecific early-stage symptoms that often lead to late detection. About 70% of cases are diagnosed at advanced stages, where survival drops to around 31%, compared to a 90% survival rate if detected early. Early diagnosis is challenging due to the cancer's asymptomatic nature and reliance on imaging techniques prone to sensitivity issues and inter-reader variability, often resulting in multiple doctor visits and diagnostic delays. These challenges highlight the potential of deep learning to assist in improving early ovarian cancer detection by providing more consistent and sensitive image interpretation.

TahHira et al.'s (2024) systematic review emphasizes the transformative potential of CNNs, particularly ResNet architectures, in analyzing diverse medical imaging like ultrasound and CT scans. CNNs excel at extracting subtle visual cues indicative of early malignancy, potentially overcoming limitations of traditional diagnostic approaches (TahHira et al., 2024).

Prior research supports this potential. Wang et al. (2022) demonstrated ResNet's effectiveness in classifying ovarian tumors in ultrasound, suggesting reduced radiologist variability, though their focus wasn't early-stage detection and interpretability was limited. Li et al. (2023) explored multimodal fusion for improved early-stage sensitivity, but complexity and lack of public implementation hinder adoption. Guha et al. (2024) highlighted transparency in AI using ResNet50 and Grad-cam for interpretable CT image classification, but with a computationally intensive architecture.



TahHira et al.'s (2024) review revealed key limitations: a focus on diagnosis over prediction/prevention, geographically limited studies, limited integrated analyses, and a critical lack of external validation (only 8.3%). Furthermore, AI explainability (AIE) remains nascent (5.2%). Addressing the critical gap in external validation (TahHira et al., 2024), our project aims to validate our data on multiple external datasets to ensure generalizability for real-world deployment. We will focus on a lightweight ResNet18 model, emphasizing AI transparency, fairness, and robustness which are essential for clinical reliability. Given the current research focus on diagnosis and the absence of work on prediction/prevention (TahHira et al.,

2024), our project specifically targets early detection.

2. Methodology

This study employed a deep learning approach to classify ovarian ultrasound images as either POCS-positive (“infected”) or PCOS-negative (“notinfected”), with the ultimate goal of improving diagnostic consistency and interpretability across ovarian conditions. We adopted a transfer learning framework using a pre-trained ResNet18 model, fine-tuned for binary classification. Our methodology involved four major stages: dataset preparation, model architecture and training, evaluation, and interpretability analysis.

2.1 Data Acquisition and Preprocessing

We used a publicly available ultrasound image dataset from Kaggle, consisting of approximately 781 infected and 1141 not infected, approximately 720 labeled images. These grayscale ultrasound images were organized into two folders: ‘infected’ (PCOS-positive) and ‘notinfected’ (PCOS-negative). The goal of preprocessing was to ensure image compatibility with the ResNet18 model and to maintain consistency across training and validation phases.

To begin, all images were resized to 224×224 pixels, matching the input size of the ResNet18 model. We also normalized the image pixel values across three channels using a mean and standard deviation of 0.5. This normalization approach was chosen to be consistent with models pretrained on ImageNet.

The dataset was then split into training and validation sets using an 80/20 ratio, ensuring stratification so that both sets preserved the same class distribution. This was done by shuffling images and copying them into their respective training or validation folders. The directory creation and image copying process was handled using the `os`, `random`, and `shutil` Python libraries.

The exact data splitting and directory setup are shown in the code snippet provided at the end of this section, which demonstrates how class folders were created and populated using a reproducible script.

```
○ project_path = "/content/drive/My Drive/ovarian-cancer-cv-project"
○ base_dir = f"{project_path}/data_set"
○ all_data_dir = os.path.join(base_dir, "all_data")
○ train_dir = os.path.join(base_dir, "train")
○ val_dir = os.path.join(base_dir, "test")
○
○ classes = ['infected', 'notinfected']
○
○ def ensure_dir(path):
○     if not os.path.exists(path):
```

```

○ os.makedirs(path)
○
○ for cls in classes:
○     ensure_dir(os.path.join(train_dir, cls))
○     ensure_dir(os.path.join(val_dir, cls))
○
○ split_ratio = 0.8
○
○ for cls in classes:
○     src_dir = os.path.join(all_data_dir, cls)
○     images = [img for img in os.listdir(src_dir) if not img.startswith('.')]
○     random.shuffle(images)
○
○     split_idx = int(len(images) * split_ratio)
○     train_images = images[:split_idx]
○     val_images = images[split_idx:]
○
○     for img in train_images:
○         src = os.path.join(src_dir, img)
○         dst = os.path.join(train_dir, cls, img)
○         shutil.copy2(src, dst)
○
○     for img in val_images:
○         src = os.path.join(src_dir, img)
○         dst = os.path.join(val_dir, cls, img)
○         shutil.copy2(src, dst)
○
○     print(f'Class '{cls}': {len(train_images)} train, {len(val_images)} validation images
copied.")

```

2.2 Model Architecture and Training

We used the ResNet18 model, pretrained on ImageNet, as our base architecture. Transfer learning was employed to adapt the model to our binary classification task. Specifically, all base layers were frozen to retain the pretrained weights that capture general visual features. Only the final fully connected layer was replaced with a new 2-node output layer, corresponding to the two classes in our dataset.

```

○ # Load pre-trained ResNet18
○ model = models.resnet18(pretrained=True)

```

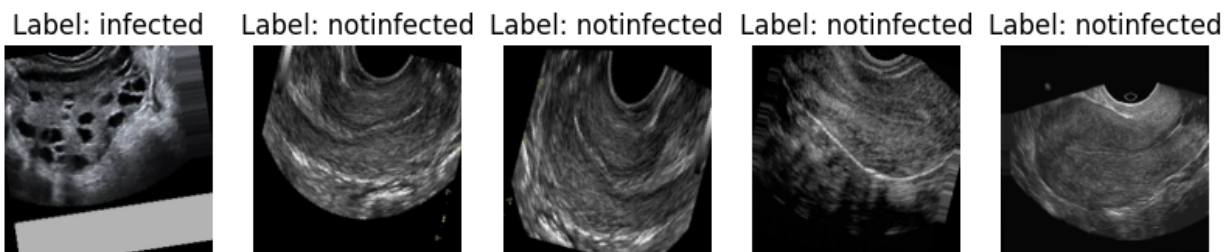
-
- `# Freeze backbone`
- `for param in model.parameters():`
- `param.requires_grad = False`
-
- `# Replace final layer for binary classification`
- `num_fts = model.fc.in_features`
- `model.fc = nn.Linear(num_fts, 2)`

The model was trained using cross-entropy loss and the Adam optimizer with a learning rate of 0.001. These choices balanced convergence speed and training stability. We performed training on a CUDA-enabled GPU when available to speed up computation.

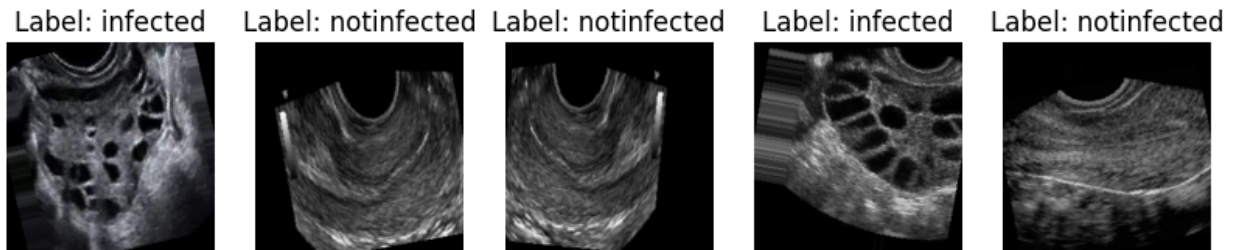
- `# Device, Loss, Optimizer`
- `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")`
- `model = model.to(device)`
- `criterion = nn.CrossEntropyLoss()`
- `optimizer = torch.optim.Adam(model.fc.parameters(), lr=0.001)`

To enhance the model's ability to generalize to new ultrasound images, we incorporated data augmentation during training. We applied random horizontal flips and random rotations up to ± 15 degrees, using PyTorch's `transforms.Compose()` module. These augmentations simulated real-world variations in image acquisition and helped prevent overfitting. Validation data was not augmented and was only resized and normalized.

Train



Test



```

○ train_transform = transforms.Compose([
○     transforms.Resize((224, 224)),
○     transforms.RandomHorizontalFlip(),
○     transforms.RandomRotation(15),
○     transforms.ToTensor(),
○     transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
○ ])
○
○ val_transform = transforms.Compose([
○     transforms.Resize((224, 224)),
○     transforms.ToTensor(),
○     transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
○ ])
○
○ train_dataset = ImageFolder(train_dir, transform=train_transform)
○ val_dataset = ImageFolder(val_dir, transform=val_transform)
○
○ train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
○ val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)

```

The training loop used DataLoader objects with a batch size of 16, and shuffled the training set each epoch. The full model definition, freezing of layers, augmentation logic, and training configuration are shown in the model training code block included later in this section.

2.3 Evaluation and Metrics

Once trained, the model was evaluated on the validation set using several standard metrics:

- Accuracy: Overall proportion of correctly classified samples.
- Precision and Recall (Sensitivity): To measure the model's performance in identifying PCOS-positive cases.
- F1 Score: The harmonic mean of precision and recall.

- ROC-AUC: To assess the classifier's ability to distinguish between classes across decision thresholds.
- Confusion Matrix: To visualize classification performance for each class.

These metrics were calculated using the sklearn.metrics library after collecting model predictions via batched inference on the validation data. The confusion matrix was plotted using Seaborn to visually summarize classification performance.

```
# Final Evaluation Metrics
    all_preds, all_labels = [], []
    model.eval()
    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.numpy())

    # Confusion Matrix
    cm = confusion_matrix(all_labels, all_preds)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=train_dataset.classes,
                yticklabels=train_dataset.classes)
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.title('Confusion Matrix')
    plt.show()

    # Classification Report
    print(classification_report(all_labels, all_preds, target_names=train_dataset.classes))
```

2.4 Interpretability via Grad-CAM

In addition to numerical metrics, we applied Grad-CAM (Gradient-weighted Class Activation Mapping) to interpret the model's decisions. Grad-CAM generates heatmaps that visualize the specific regions in the ultrasound image that contributed most to the predicted class.

We targeted the last convolutional layer of the ResNet18 model (layer4[-1].conv2) using forward and backward hooks to extract activations and gradients. These were then combined to compute a class-specific saliency map.

The Grad-CAM heatmap was overlaid on the original input image using OpenCV's color maps and matplotlib for display. This helped assess whether the model was focusing on clinically meaningful features, such as follicular structures.

The code block at the end of this section includes the Grad-CAM implementation and an example of how it was applied to a sample from the validation set.

```
○ def gradcam(model, image_tensor, class_idx=None):
○     model.eval()
○     activations = []
○     gradients = []
○
○     # Forward hook
○     def forward_hook(module, input, output):
○         print("Forward hook triggered.") # Debug
○         activations.append(output.detach())
○
○     # Backward hook (with explicit checks)
○     def backward_hook(module, grad_input, grad_output):
○         print("Backward hook triggered. Grad output:", grad_output) # Debug
○         if grad_output and grad_output[0] is not None:
○             gradients.append(grad_output[0].detach())
○
○     # Target the last convolutional layer (adjust for your model)
○     target_layer = model.layer4[-1].conv2 # Example for ResNet
○     handle_forward = target_layer.register_forward_hook(forward_hook)
○     handle_backward = target_layer.register_backward_hook(backward_hook)
○
○     # Forward pass (ensure gradients)
○     image_tensor = image_tensor.unsqueeze(0).to(device)
○     image_tensor.requires_grad_(True) # Critical!
○     output = model(image_tensor)
○
○     # Class index
○     if class_idx is None:
○         class_idx = torch.argmax(output, dim=1).item()
○
```

```

○ # Backward pass (force gradient flow)
○ model.zero_grad()
○ score = output[0, class_idx]
○ score.backward(retain_graph=True)
○
○ # Check hooks
○ if not gradients:
○     handle_forward.remove()
○     handle_backward.remove()
○     raise RuntimeError(
○         "No gradients captured. Possible fixes:\n"
○         "1. Target a different layer (e.g., model.layer3[-1].conv2).\n"
○         "2. Ensure image_tensor.requires_grad=True.\n"
○         "3. Debug hooks via print statements."
○     )
○
○ # Compute CAM
○ activation = activations[0].squeeze(0).cpu()
○ gradient = gradients[0].squeeze(0).cpu()
○ weights = gradient.mean(dim=(1, 2))
○ cam = torch.zeros(activation.shape[1:], dtype=torch.float32)
○ for i, w in enumerate(weights):
○     cam += w * activation[i]
○ cam = torch.clamp(cam, min=0)
○ cam = (cam - cam.min()) / (cam.max() + 1e-8)
○ cam = cv2.resize(cam.numpy(), (224, 224))
○
○ # Cleanup
○ handle_forward.remove()
○ handle_backward.remove()
○
○ return cam
○
○
○ def show_gradcam(image_tensor, cam, title="Grad-CAM"):
○     img = image_tensor.permute(1, 2, 0).cpu().numpy()
○     img = (img * 0.5 + 0.5)
○     img = np.clip(img, 0, 1)
○
○     heatmap = cv2.applyColorMap(np.uint8(255 * cam), cv2.COLORMAP_JET)

```



```

○ heatmap = np.float32(heatmap) / 255
○ heatmap = heatmap[..., ::-1] # BGR to RGB
○
○ overlay = heatmap * 0.4 + img
○ overlay = np.clip(overlay, 0, 1)
○
○ plt.imshow(overlay)
○ plt.title(title)
○ plt.axis('off')
○ plt.show()
○
○ # Run Grad-CAM on validation sample
○ images, labels = next(iter(val_loader))
○ image_tensor = images[5]
○ cam = gradcam(model, image_tensor)
○ imshow_gradcam(image_tensor, cam, title=f"Grad-CAM Example, True:
{train_dataset.classes[labels[5]]}")

```

3. Experimental Results

We split our dataset into split into

Train files: 1536

Val files: 386

And had a 100% validation accuracy which suggests that either the dataset is too simple. We checked for overlapping files between the two however we got the following:

Epoch [1/5], Loss: 0.3061, Accuracy: 0.8919

Validation Accuracy: 0.9974

Epoch [2/5], Loss: 0.0924, Accuracy: 0.9928

Validation Accuracy: 1.0000

Epoch [3/5], Loss: 0.0578, Accuracy: 0.9941

Validation Accuracy: 1.0000

Epoch [4/5], Loss: 0.0472, Accuracy: 0.9941

Validation Accuracy: 1.0000

Epoch [5/5], Loss: 0.0306, Accuracy: 0.9967

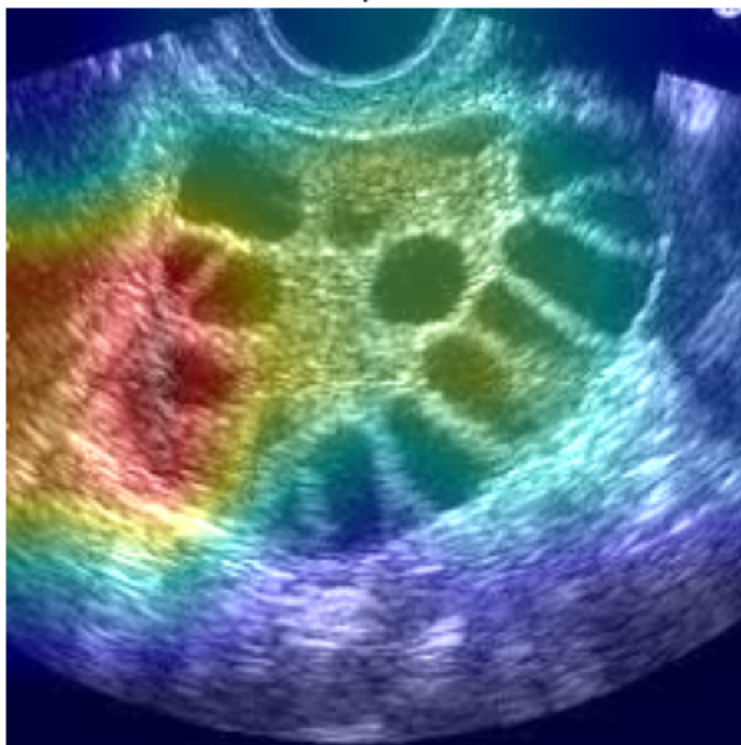
Validation Accuracy: 1.0000

	precision	recall	f1-score	support
infected	1.00	1.00	1.00	157
notinfected	1.00	1.00	1.00	229

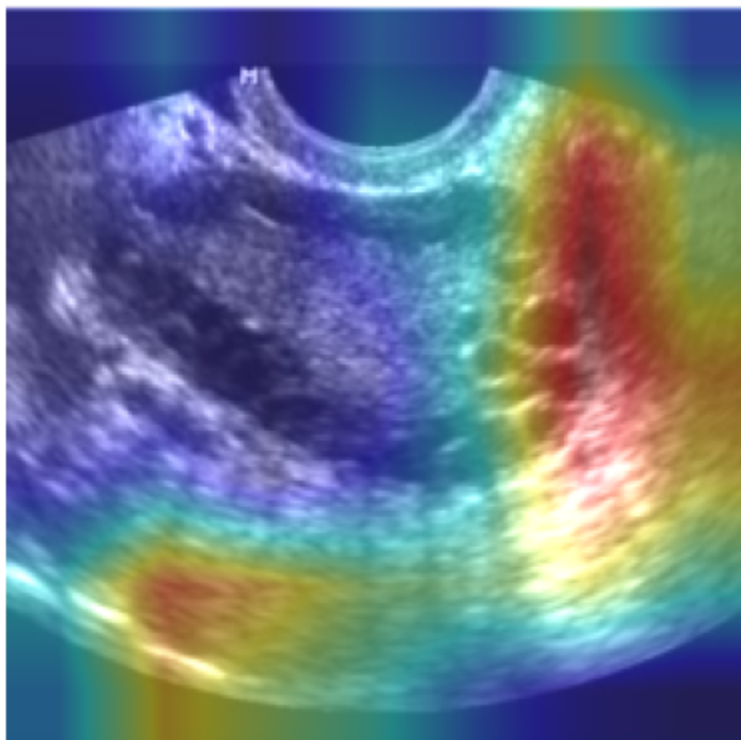
accuracy			1.00	386
macro avg	1.00	1.00	1.00	386
weighted avg	1.00	1.00	1.00	386

In several correctly classified PCOS-positive examples, the Grad-CAM heatmaps highlighted regions consistent with polycystic morphology—specifically around the ovarian periphery where follicular structures are typically visible.

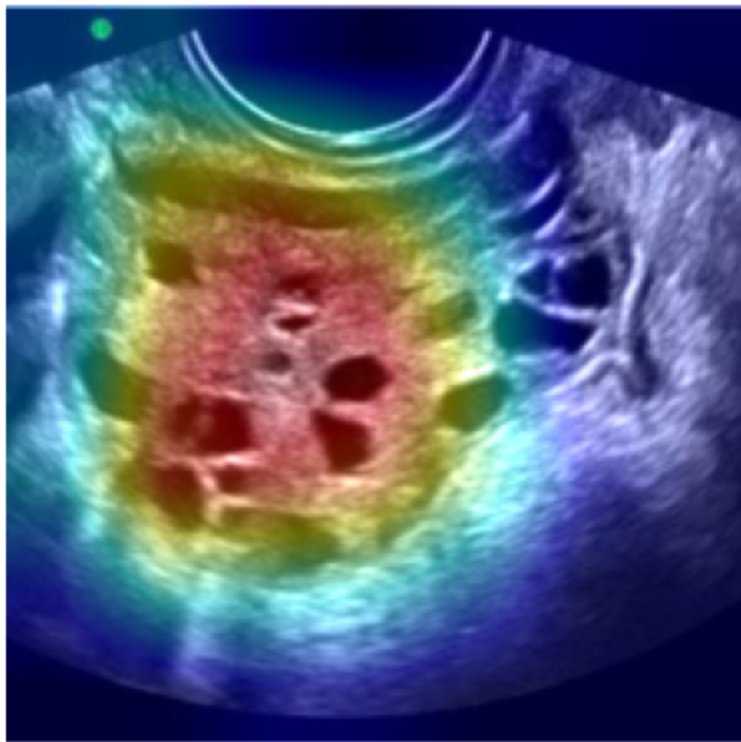
Grad-CAM Example, True: infected



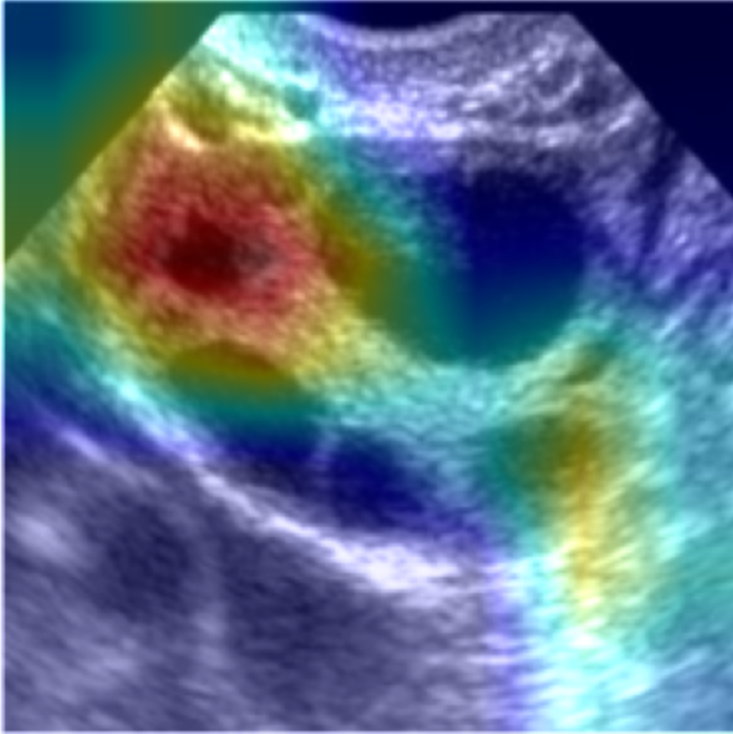
Grad-CAM Example, True: infected



Grad-CAM Example, True: infected



Grad-CAM Example, True: infected



4. Discussion

Our original goal was to explore whether a lightweight convolutional neural network could be trained to improve early-stage ovarian cancer detection across multiple imaging modalities, including ultrasound and CT, using datasets such as MMOTU and TCGA-OV. However, due to limited access to curated multimodal data, annotation inconsistencies, and time constraints, we narrowed our focus to a single-modality classification task: detecting polycystic ovary syndrome (PCOS) from ultrasound images using a clearly labeled Kaggle dataset. This pivot allowed us to design, train, and evaluate a complete, reproducible pipeline using real-world clinical data while maintaining our core emphasis on interpretability, generalization, and deployment feasibility.

4.1 Why We Abandoned MMOTU

Although the MMOTU dataset appeared promising, we discovered that it exists in multiple versions from different research, each with different annotation formations and processing states. Furthermore, while the dataset contained image-level annotations, the label structure was poorly documented, and we were unable to confidently map image file names or folder structures to the diagnostic categories described in the associated research paper. Specifically, the image annotations and abbreviations did not match the abbreviations and descriptions in the MMOTU

paper. This made it unclear whether a given scan represented a benign tumor, a malignant tumor, or another case.

As a result, we were unable to extract reliable ground-truth tables from MMOTU, which made it impossible to compute any meaningful evaluation metrics such as accuracy, F1 score, confusion matrix, or ROC-AUC. Since these metrics rely on having one label per image, and we could not validate the annotations, we made the decision to exclude MMOTU from our testing workflow to preserve the integrity of our results.

4.2 Simplifications and Assumptions

Our study made several assumptions and simplifications:

- We assumed the dataset’s binary PCOS labels were accurate representations of clinical diagnosis, though no histological or clinical validation was provided.
- We used only static 2D grayscale images, excluding dynamic scan sequences or color Doppler information.
- We did not include patient metadata (e.g., age, BMI, hormone levels), which are important to real-world diagnosis.
- We assumed transfer learning from ImageNet-trained models would generalize well to ultrasound, despite the domain shift from natural to medical images.

These simplifications allowed us to scope the project effectively, but limited its clinical readiness.

4.3 Limitations

The most pressing limitation is the lack of large, diverse, and well-labeled publicly available datasets for ovarian disease detection. Although we explored multiple sources, very few were openly accessible, clearly annotated, and suitable for supervised learning. This limited the scope of our experiments and forced us to rely on a single, relatively small dataset (~1932 images), reducing our ability to test generalization across clinical populations and imaging devices.

Another limitation involved our initial interest in using 3D ultrasound volumes, which often provide more anatomical context than 2D frames. However, we found that the available datasets stored 3D scans in .vtk (Visualization Toolkit) format, and we were unsure how to extract 2D slices or projections with labeled regions that indicated whether a volume was “infected” or “notinfected.” The lack of accessible documentation on the labeling structure of these volumes prevented us from converting the files into usable 2D .png or .jpg images with confirmed class annotations. As a result, we excluded these 3D volumes to avoid introducing mislabeled data.

We also did not complete our original plan of validating on CT scans from the TCGA-OV dataset, due to the time required for preprocessing large volumes of DICOM data and the lack of direct label mappings for PCOS detection. We also did not integrate multimodal inputs (e.g., biomarkers or patient history) that would have provided additional context to improve diagnostic accuracy and better reflect real-world clinical decision-making.

5. Conclusion

This project aimed to support the early detection of ovarian conditions using deep learning and medical ultrasound data. While we initially planned to work with multiple datasets and imaging modalities, including MMOTU and CT scans, we narrowed our scope to the Kaggle PCOS ultrasound dataset due to challenges related to unclear annotations, limited documentation, and time constraints. This allowed us to complete a well-defined, reproducible binary classification pipeline centered around interpretability and practical deployment.

Looking ahead, we plan to resolve labeling inconsistencies in the MMOTU dataset through direct collaboration or systematic verification. We also hope to expand our work to include larger and more diverse datasets from multiple institutions, which would help improve the generalizability of our findings. Processing 3D ultrasound volumes and extracting usable 2D slices from .vtk files is another key direction we aim to pursue. In addition, integrating multimodal information, such as CA-125 biomarker data and patient metadata, could enhance diagnostic accuracy and bring the model closer to real-world clinical application. We also plan to experiment with alternative interpretability methods, including SHAP and LIME, to offer more granular insights into the model's predictions. Finally, implementing domain adaptation techniques would allow us to better generalize across different scanners, imaging conditions, and healthcare settings.

These future steps will help extend the impact of this work and advance the development of accessible, trustworthy AI tools for ovarian disease detection.